BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN

□□□□

[Cần bổ sung loại báo cáo]

ỨNG DỤNG ARANGODB XÂY DỰNG HỆ THỐNG GỢI Ý VIỆC LÀM VÀ ĐƯỜNG KẾT NỐI GIỚI THIỆU

Ngành: [Cần bổ sung]

SINH VIÊN THỰC HIỆN:

1. [Cần bổ sung MSSV – Họ tên – Lớp]
2. [Cần bổ sung MSSV – Họ tên – Lớp]
3. [Cần bổ sung MSSV – Họ tên – Lớp]

TP. HỒ CHÍ MINH, tháng 10 năm 2026

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN

□□□□

[Cần bổ sung loại báo cáo]

ỨNG DỤNG ARANGODB XÂY DỰNG HỆ THỐNG GỢI Ý VIỆC LÀM VÀ ĐƯỜNG KẾT NỐI GIỚI THIỆU

Ngành: [Cần bổ sung]

GIẢNG VIÊN HƯỚNG DẪN:

[Cần bổ sung họ tên giảng viên hướng dẫn]

SINH VIÊN THỰC HIỆN:

1. [Cần bổ sung MSSV – Họ tên – Lớp]
2. [Cần bổ sung MSSV – Họ tên – Lớp]
3. [Cần bổ sung MSSV – Họ tên – Lớp]

TP. HỒ CHÍ MINH, tháng 10 năm 2026

LỜI CẢM ƠN

Nhóm thực hiện xin trân trọng cảm ơn Nhà trường và Khoa Công nghệ Thông tin đã tạo điều kiện học tập, nghiên cứu để thực hiện đề tài. Nhóm xin cảm ơn giảng viên hướng dẫn đã hỗ trợ định hướng chuyên môn và góp ý cho quá trình hoàn thiện báo cáo.

Nhóm cũng xin cảm ơn các tác giả và cộng đồng phát triển ArangoDB, FastAPI, React cùng các công cụ mã nguồn mở đã cung cấp tài liệu và nền tảng kỹ thuật phục vụ việc xây dựng ứng dụng. Báo cáo còn những hạn chế về dữ liệu đánh giá và phạm vi thử nghiệm; nhóm mong nhận được góp ý để tiếp tục hoàn thiện.

TP. Hồ Chí Minh, tháng 10 năm 2026

Nhóm thực hiện

MỤC LỤC

LỜI CẢM ƠN i
DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT v
DANH MỤC HÌNH vi
DANH MỤC BẢNG vii
MỞ ĐẦU 1
1. GIỚI THIỆU 1
2. MỤC TIÊU ĐỀ TÀI 1
3. ĐỐI TƯỢNG VÀ PHẠM VI ĐỀ TÀI 2
4. PHƯƠNG PHÁP THỰC HIỆN VÀ NGUỒN BẰNG CHỨNG 2
CHƯƠNG 1: KHẢO SÁT HỆ THỐNG 3
1.1. MỤC TIÊU HỆ THỐNG 3
1.2. HIỆN TRẠNG VÀ NHU CẦU 3
1.2.1. Hiện trạng tổ chức 3
1.2.2. Quy trình nghiệp vụ 3
1.2.3. Vấn đề cần giải quyết 4
1.3. CƠ SỞ LÝ THUYẾT VỀ ARANGODB 4
1.3.1. Document và property graph 4
1.3.2. Named graph và chiều duyệt 4
1.3.3. AQL và graph traversal 4
1.3.4. Chỉ mục, transaction và giới hạn truy vấn 5
1.4. CƠ SỞ LÝ THUYẾT VỀ HỆ GỢI Ý 5
1.5. LÝ DO LỰA CHỌN GIẢI PHÁP 5
CHƯƠNG 2: PHÂN TÍCH HỆ THỐNG 7
2.1. TÁC NHÂN VÀ PHÂN QUYỀN 7
2.2. YÊU CẦU CHỨC NĂNG 8
2.3. YÊU CẦU PHI CHỨC NĂNG 9

2.4. PHÂN TÍCH CÁC CA SỬ DỤNG 9
2.4.1. Xem công việc được gợi ý 9
2.4.2. Tìm đường tới người giới thiệu 9
2.4.3. Gửi và xử lý yêu cầu giới thiệu 10
2.5. QUY TẮC NGHIỆP VỤ VÀ TRƯỜNG HỢP BIÊN 10
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG 12
3.1. KIẾN TRÚC TỔNG THỂ 12
3.1.1. Phân tách module 12
3.2. THIẾT KẾ DỮ LIỆU ARANGODB 13
3.2.1. Collections thực thể và nghiệp vụ 13
3.2.2. Edge collections và chiều quan hệ 14
3.2.3. Schema, migration và chỉ mục 15
3.3. THIẾT KẾ THUẬT TOÁN GỢI Ý VIỆC LÀM 15
3.3.1. Đầu vào và tiền xử lý 15
3.3.2. Công thức điểm tổng hợp 16
3.3.3. Ví dụ tính điểm và xếp hạng 17
3.3.4. Dữ liệu thiếu và nguồn công khai 17
3.4. THIẾT KẾ TÌM VÀ XẾP HẠNG ĐƯỜNG GIỚI THIỆU 17
3.4.1. Duyệt graph có ràng buộc 17
3.4.2. Điểm đường và đường ngắn nhất 18
3.4.3. Kết hợp việc làm và mạng lưới 18
3.5. THIẾT KẾ API, GIAO DIỆN VÀ TÍNH NHẤT QUÁN 18
CHƯƠNG 4: CÀI ĐẶT HỆ THỐNG 20
4.1. MÔI TRƯỜNG VÀ CÔNG NGHỆ 20
4.2. CÀI ĐẶT DATABASE VÀ KHỞI TẠO 20
4.3. HIỆN THỰC AQL VÀ MATCHING 21
4.4. HIỆN THỰC GIAO DIỆN VÀ LUỒNG NGƯỜI DÙNG 22

4.5. DỮ LIỆU CÔNG KHAI VÀ NGUỒN GỐC 24
4.6. BẢO MẬT, GHI NHẬN VÀ VẬN HÀNH CỤC BỘ 25
CHƯƠNG 5: THỬ NGHIỆM VÀ TRIỂN KHAI 26
5.1. PHƯƠNG PHÁP VÀ MÔI TRƯỜNG THỬ NGHIỆM 26
5.2. KẾT QUẢ KIỂM THỬ CHỨC NĂNG 26
5.3. CHỨNG MINH ĐÓNG GÓP CỦA ARANGODB 27
5.4. ĐÁNH GIÁ HIỆU NĂNG TRUY VẤN 28
5.5. TRIỂN KHAI VÀ PHỤC HỒI 29
5.6. ĐỐI CHIẾU MỤC TIÊU VÀ GIỚI HẠN ĐÁNH GIÁ 29
5.7. KỊCH BẢN TRÌNH DIỄN KHI BẢO VỆ 30
KẾT LUẬN 32
1. KẾT QUẢ ĐẠT ĐƯỢC 32
2. ĐÁNH GIÁ KẾT QUẢ 32
3. HẠN CHẾ 32
4. HƯỚNG PHÁT TRIỂN 33
TÀI LIỆU THAM KHẢO 34
PHỤ LỤC: HƯỚNG DẪN TÁI LẬP VÀ ĐỐI CHIẾU 35
A. KIỂM TRA CHẠY CỤC BỘ 35
B. BẢN ĐỒ SOURCE VÀ BẰNG CHỨNG 35
C. CÁC THÔNG TIN HÀNH CHÍNH CẦN HOÀN THIỆN 36

DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Viết tắt	Tiếng Anh	Tiếng Việt
AQL	ArangoDB Query Language	Ngôn ngữ truy vấn ArangoDB
API	Application Programming Interface	Giao diện lập trình ứng dụng
ATS	Applicant Tracking System	Hệ thống quản lý ứng viên
DBMS	Database Management System	Hệ quản trị cơ sở dữ liệu
DTO	Data Transfer Object	Đối tượng truyền dữ liệu
HTTP	Hypertext Transfer Protocol	Giao thức truyền siêu văn bản
JWT	JSON Web Token	Token xác thực dạng JSON
MVP	Minimum Viable Product	Sản phẩm khả dụng tối thiểu
NDCG	Normalized Discounted Cumulative Gain	Chỉ số chất lượng thứ hạng chuẩn hóa
RPO / RTO	Recovery Point / Time Objective	Mục tiêu điểm/thời gian phục hồi
SLA	Service Level Agreement	Cam kết mức dịch vụ
TLS	Transport Layer Security	Bảo mật lớp truyền tải
UUID	Universally Unique Identifier	Định danh duy nhất

DANH MỤC HÌNH

Hình 2.1: Các nhóm chức năng theo tác nhân 7

Hình 2.2: Vòng đời yêu cầu giới thiệu trong MVP 10

Hình 3.1: Kiến trúc ứng dụng và luồng dữ liệu 12

Hình 3.2: Mô hình graph cốt lõi của Job Referral Network 14

Hình 4.1: Khu vực gợi ý việc làm trên dashboard với dữ liệu kiểm thử 23

Hình 4.2: Giao diện đường giới thiệu A → B → C trên fixture 24

Hình 5.1: So sánh p50 và p95 truy vấn trên benchmark synthetic local 28

DANH MỤC BẢNG

Bảng 2.1: Tác nhân và quyền nghiệp vụ chính 7
Bảng 2.2: Yêu cầu và tiêu chí nghiệm thu cốt lõi 8
Bảng 3.1: Các collections chính 13
Bảng 3.2: Định nghĩa các quan hệ đồ thị 14
Bảng 3.3: Một số chỉ mục quan trọng 15
Bảng 3.4: Thành phần và trọng số điểm việc làm 16
Bảng 3.5: Các API chính cho gợi ý và graph 18
Bảng 4.1: Cấu hình công nghệ theo mã nguồn 20
Bảng 5.1: Tổng hợp kết quả kiểm thử lưu trong dự án 26
Bảng 5.2: Thay đổi dữ liệu và kết quả graph quan sát được 27
Bảng 5.3: Thời gian truy vấn trong benchmark local 28
Bảng 5.4: Mức đáp ứng mục tiêu đề tài 29
Bảng PL.1: Đối chiếu nội dung báo cáo với source/artifact 35

MỞ ĐẦU

1. GIỚI THIỆU

Thông tin tuyển dụng thường mô tả vị trí, kỹ năng và kinh nghiệm cần thiết, nhưng một ứng viên còn phải trả lời câu hỏi mình có thể tiếp cận doanh nghiệp thông qua ai. Hai bài toán "công việc nào phù hợp" và "đường kết nối nào dẫn tới người giới thiệu" sử dụng các loại dữ liệu khác nhau: thuộc tính của hồ sơ và tin tuyển dụng, cùng các quan hệ giữa người dùng, kỹ năng và doanh nghiệp. Đề tài lựa chọn kết hợp hai bài toán trong một ứng dụng có thể chạy và kiểm chứng.

ArangoDB được chọn làm cơ sở dữ liệu cốt lõi để lưu các thực thể dưới dạng document và các quan hệ dưới dạng edge. Thay vì chỉ lưu danh sách công việc rồi lọc theo từ khóa, hệ thống sử dụng AQL để đối chiếu hồ sơ, tính điểm phù hợp và duyệt mạng lưới kết nối. Kết quả được đưa lên giao diện để ứng viên hiểu lý do gợi ý và có thể thực hiện yêu cầu giới thiệu.

Đề tài mang tên "Ứng dụng ArangoDB xây dựng hệ thống gợi ý việc làm và đường kết nối giới thiệu". Sản phẩm trong mã nguồn có tên Job Referral Network, giao diện sử dụng nhận diện "nối.". Báo cáo trình bày theo hướng ứng dụng: từ khảo sát bài toán, phân tích yêu cầu, thiết kế dữ liệu và thuật toán đến cài đặt, thử nghiệm và triển khai cục bộ.

2. MỤC TIÊU ĐỀ TÀI

Mục tiêu thứ nhất là tìm hiểu cách sử dụng document collections, edge collections, named graph, AQL, chỉ mục và transaction của ArangoDB trong một bài toán có dữ liệu quan hệ. Mục tiêu thứ hai là xây dựng ứng dụng hoàn chỉnh gồm giao diện, API và cơ sở dữ liệu thật, trong đó người dùng cập nhật hồ sơ, xem việc làm được gợi ý, tìm đường kết nối và theo dõi yêu cầu giới thiệu.

Mục tiêu thứ ba là làm cho kết quả có thể giải thích và kiểm chứng. Điểm phù hợp phải được phân rã theo các thành phần; đường đi phải xuất phát từ các cạnh được lưu trong cơ sở dữ liệu và tuân thủ quyền riêng tư. Việc thay đổi kỹ năng, trạng thái cạnh hoặc thông tin việc làm phải làm thay đổi kết quả đọc lại. Mục tiêu cuối cùng là cung cấp bằng chứng kiểm thử, giới hạn áp dụng và hướng phát triển, tránh đồng nhất một bản chạy cục bộ với hệ thống vận hành quy mô lớn.

3. ĐỐI TƯỢNG VÀ PHẠM VI ĐỀ TÀI

Đối tượng nghiên cứu gồm mô hình đồ thị quan hệ nghề nghiệp, truy vấn AQL, phương pháp gợi ý theo nội dung có trọng số, cách xếp hạng đường giới thiệu và quy trình xử lý yêu cầu giới thiệu. Đối tượng sử dụng gồm ứng viên, người giới thiệu, người quản lý tuyển dụng của doanh nghiệp và quản trị viên. Một người dùng có thể đồng thời tham gia nhiều vai trò nghiệp vụ.

Phạm vi triển khai là MVP trên một máy chủ ArangoDB, giao diện tiếng Việt, API FastAPI và frontend React. Duyệt mạng lưới được giới hạn tối đa ba cạnh kết nối người dùng. Hệ thống gợi ý theo quy tắc và trọng số, chưa huấn luyện mô hình học máy, chưa dự đoán xác suất trúng tuyển. Dữ liệu công khai là ảnh chụp danh mục tại ngày thu thập; dữ liệu quan hệ để thử nghiệm sử dụng các hồ sơ và doanh nghiệp được gắn nhãn kiểm thử.

Ngoài phạm vi hiện tại là tải lên CV, gửi email, chuyển hồ sơ tới ATS, tìm kiếm ứng viên dành cho nhà tuyển dụng, đánh giá thuật toán bằng phản hồi tuyển dụng thực tế và vận hành cluster. Các chức năng này được nêu như hướng phát triển, không được tính là kết quả đã hoàn thành.

4. PHƯƠNG PHÁP THỰC HIỆN VÀ NGUỒN BẰNG CHỨNG

Báo cáo được xây dựng bằng cách đối chiếu yêu cầu trong detai.md, mã nguồn tại backend, database và frontend, tài liệu thiết kế trong docs, cùng các báo cáo máy đọc được tại docs/reports. Các số liệu kiểm thử được lấy từ JUnit XML hoặc JSON, không tự suy ra từ số lượng file kiểm thử. Báo cáo chốt theo trạng thái dự án được kiểm tra ngày 12/10/2026; các kết quả chạy trước đó được ghi rõ nguồn và thời điểm, không mô tả là phép đo mới của lần biên soạn này.

Phương pháp đánh giá gồm kiểm thử tích hợp trên ArangoDB thật, kiểm thử giao diện qua HTTP thật, thay đổi dữ liệu để quan sát kết quả truy vấn, kiểm tra EXPLAIN/PROFILE và thử phục hồi bản sao lưu. Những phép thử này chứng minh tính đúng đắn trong phạm vi kịch bản đã chạy. Chúng chưa thay thế đánh giá mức độ hài lòng của người dùng hoặc kiểm thử tải đồng thời ở môi trường sản xuất.

CHƯƠNG 1: KHẢO SÁT HỆ THỐNG

1.1. MỤC TIÊU HỆ THỐNG

Hệ thống hướng tới việc giúp ứng viên tìm được các công việc phù hợp với dữ liệu hồ sơ đã khai báo và nhận biết ai trong mạng lưới có thể giới thiệu tới doanh nghiệp. Kết quả gợi ý phải trả lời được ba nội dung: mức độ khớp với công việc, những điều kiện còn thiếu và đường kết nối hợp lệ tới người giới thiệu. Khi ứng viên chọn một cơ hội, hệ thống hỗ trợ gửi yêu cầu và theo dõi các bước xử lý.

Một ví dụ nghiệp vụ là ứng viên A có kỹ năng Python và SQL, tìm công việc tại doanh nghiệp X. A kết nối với B, B kết nối với C; C có quan hệ việc làm hiện tại đã được xác minh tại X. Hệ thống đối chiếu kỹ năng của A với yêu cầu của công việc, sau đó tìm đường A → B → C. Việc A và C cùng có một kỹ năng không tự tạo thành quan hệ kết nối xã hội.

1.2. HIỆN TRẠNG VÀ NHU CẦU

1.2.1. Hiện trạng tổ chức

Đề tài sử dụng mô hình nền tảng tuyển dụng và giới thiệu việc làm giả định để thiết kế ứng dụng; chưa có khảo sát phỏng vấn một doanh nghiệp triển khai cụ thể. Trong mô hình này, ứng viên quản lý hồ sơ cá nhân, doanh nghiệp quản lý tin tuyển dụng, người giới thiệu xác nhận việc hỗ trợ và quản trị viên kiểm soát danh mục, tài khoản, xác minh quyền doanh nghiệp.

Dữ liệu đầu vào có hai nhóm. Nhóm thứ nhất là tin tuyển dụng công khai có nguồn để minh họa việc tìm kiếm và đối chiếu kỹ năng. Nhóm thứ hai là dữ liệu quan hệ kiểm thử để kiểm tra duyệt đồ thị và quy trình giới thiệu. Phân biệt hai nhóm giúp tránh sử dụng nhân viên giả lập như bằng chứng về quan hệ thật tại các công ty được thu thập tin.

1.2.2. Quy trình nghiệp vụ

Trong quy trình của ứng dụng, ứng viên đăng ký, cập nhật hồ sơ và chọn kỹ năng từ danh mục chuẩn hóa. Hệ thống đọc các cạnh has_skill để xếp hạng công việc còn mở. Ứng viên có thể lọc kết quả theo từ khóa, địa điểm, doanh nghiệp, cấp độ hoặc hình thức làm việc và mở chi tiết để xem phần giải thích.

Khi tìm đường giới thiệu, hệ thống duyệt những kết nối đã được chấp nhận, kiểm tra trạng thái tài khoản và quyền xuất hiện trên đường đi. Người cuối đường

phải đang làm việc tại doanh nghiệp và có employment đã xác minh. Ứng viên gửi yêu cầu; người nhận có thể chấp nhận hoặc từ chối. Sau khi chấp nhận, người giới thiệu ghi xác nhận đã giới thiệu kèm mã bằng chứng, rồi hoàn tất với kết quả. Mỗi lần chuyển trạng thái được lưu lịch sử.

1.2.3. Vấn đề cần giải quyết

Một danh sách việc làm theo thời gian đăng không thể hiện mức độ phù hợp riêng của từng hồ sơ. Một đồ thị chỉ vẽ trên giao diện không đủ chứng minh đường kết nối được tìm từ dữ liệu thật. Một đường đi được tìm ở thời điểm trước cũng không bảo đảm còn hợp lệ khi gửi yêu cầu, vì quan hệ hoặc quyền riêng tư có thể thay đổi. Thiết kế cần xử lý đồng thời cá nhân hóa, nguồn dữ liệu, tính hợp lệ của quan hệ và kiểm tra lại nghiệp vụ tại thời điểm ghi.

1.3. CƠ SỞ LÝ THUYẾT VỀ ARANGODB

1.3.1. Document và property graph

ArangoDB biểu diễn graph bằng các document của đỉnh và cạnh. Cạnh có thuộc tính _from và _to để tham chiếu đỉnh; collection cạnh phân loại quan hệ. Mô hình này phù hợp khi cần lưu cả thuộc tính thực thể và thuộc tính quan hệ, ví dụ trạng thái kết nối hoặc thời gian làm việc [1]. Trong đề tài, hồ sơ là dữ liệu document, còn việc người dùng có một kỹ năng hay làm tại một công ty được lưu bằng edge.

1.3.2. Named graph và chiều duyệt

Named graph khai báo các collections và quan hệ cho phép giữa các loại đỉnh. OUTBOUND đi theo chiều cạnh, INBOUND đi ngược chiều, ANY sử dụng cả hai chiều [1]. Dự án chọn ANY cho connections để một kết nối được chấp nhận có thể sử dụng từ hai đầu, trong khi vẫn chỉ lưu một cạnh cho một cặp người dùng. Quan hệ works_at và has_skill giữ chiều người dùng tới công ty hoặc kỹ năng.

Việc khai báo named graph không tự bảo đảm mọi thao tác trực tiếp lên document đều có kiểm tra toàn vẹn như graph API [1]. Vì dự án có các thao tác repository trực tiếp, ứng dụng còn kiểm tra tồn tại, loại đỉnh, quyền và tính nhất quán của quan hệ. Đây là trách nhiệm bổ sung của thiết kế triển khai.

1.3.3. AQL và graph traversal

AQL là ngôn ngữ khai báo để đọc và thay đổi dữ liệu trong ArangoDB; việc tạo collection, index hoặc database sử dụng các giao diện quản trị thay vì câu

lệnh AQL [2]. Dự án dùng AQL cho lọc tin, đối chiếu kỹ năng, tính điểm và duyệt đường kết nối; migration sử dụng driver để thiết lập schema.

Traversal trong AQL cho phép xác định độ sâu, chiều duyệt, đỉnh, cạnh và đường đi. PRUNE dừng mở rộng nhánh khi điều kiện không hợp lệ xuất hiện [3]. Hệ thống kết hợp PRUNE với FILTER để vừa dừng nhánh vừa loại đỉnh không hợp lệ khỏi kết quả. Truy vấn giới hạn collections được duyệt để không đi vòng qua kỹ năng hoặc công ty và tạo ra kết nối xã hội sai.

1.3.4. Chỉ mục, transaction và giới hạn truy vấn

Trong triển khai của dự án, primary index hỗ trợ định danh document; edge index hỗ trợ đọc hai đầu quan hệ; persistent indexes phục vụ lọc trạng thái và tìm nhân viên theo doanh nghiệp. Unique indexes bảo vệ các quy tắc như một cạnh cho một cặp người dùng, một công ty cho một job và một bộ ba ứng viên–người giới thiệu–công việc cho yêu cầu giới thiệu.

Các thao tác gồm nhiều bản ghi được thực hiện trong transaction. Ví dụ chuyển trạng thái referral đồng thời ghi lịch sử, notification và audit; lỗi bất kỳ làm toàn bộ thao tác được hủy. Repository giới hạn truy vấn AQL ở 5 giây và 64 MiB bộ nhớ, đồng thời giới hạn depth, offset và limit. Đây là cấu hình kiểm soát tài nguyên của dự án, không phải khẳng định rằng mọi truy vấn ArangoDB đều có các giới hạn đó.

1.4. CƠ SỞ LÝ THUYẾT VỀ HỆ GỢI Ý

Hệ thống gợi ý theo nội dung đối chiếu đặc trưng của đối tượng cần gợi ý với thông tin người dùng. Trong bài toán này, đối tượng là công việc; đặc trưng gồm kỹ năng, kinh nghiệm, địa điểm, cấp độ và hình thức làm việc. Thiết kế của đề tài sử dụng quy tắc minh bạch để mỗi điểm thành phần có thể giải thích bằng dữ liệu, không yêu cầu lịch sử đánh giá công việc của nhiều người dùng.

Gợi ý đường kết nối là bài toán khác với độ phù hợp nghề nghiệp. Một công việc có thể phù hợp nhưng chưa có người giới thiệu trong mạng lưới; một đường đi ngắn có thể dẫn tới công việc mà ứng viên thiếu kỹ năng. Do đó hệ thống tách gợi ý việc làm và cơ hội có đường giới thiệu. Với cơ hội có đường, điều kiện năng lực được kiểm tra trước, điểm việc làm được ưu tiên trước điểm đường đi.

1.5. LÝ DO LỰA CHỌN GIẢI PHÁP

ArangoDB cho phép dự án lưu các document nghiệp vụ và biểu diễn quan hệ bằng edges trong cùng một cơ sở dữ liệu, đồng thời thực hiện traversal bằng

AQL. Đây là sự phù hợp về mô hình với đề tài, không phải kết luận ArangoDB luôn nhanh hơn cơ sở dữ liệu quan hệ. Các hệ quản trị khác cũng có thể giải quyết bài toán; báo cáo chưa xây dựng phép đo so sánh công bằng với chúng.

Đổi lại, việc thiết kế chiều cạnh, trạng thái và quyền riêng tư làm truy vấn phức tạp hơn. Duyệt nhiều nhánh khiến chi phí tăng theo độ sâu. Dự án chọn modular monolith và single server để tập trung vào tính đúng đắn; đánh giá cluster hoặc lựa chọn hệ quản trị ở quy mô lớn cần một nghiên cứu bổ sung.

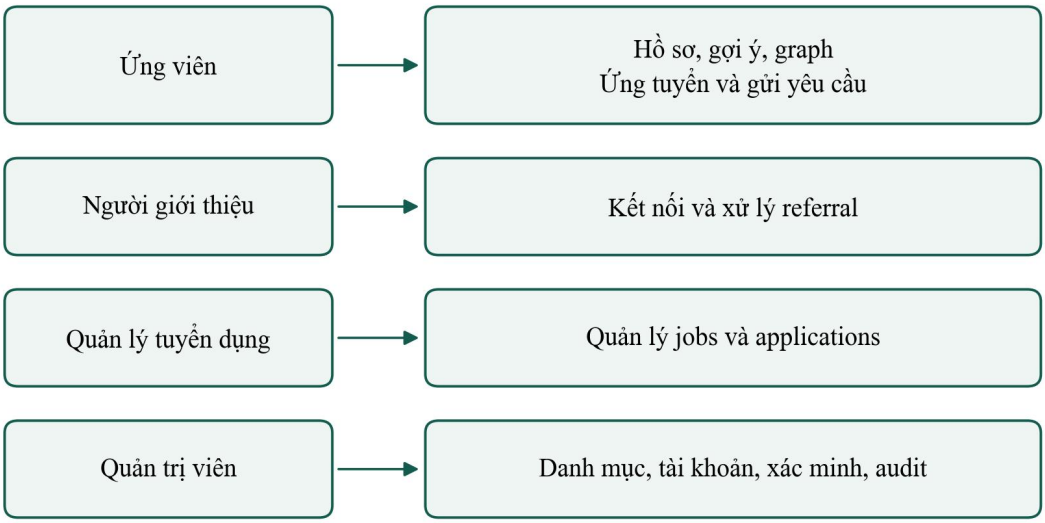
CHƯƠNG 2: PHÂN TÍCH HỆ THỐNG

2.1. TÁC NHÂN VÀ PHÂN QUYỀN

Các chức năng được tổ chức theo tác nhân nhưng tài khoản không bị giới hạn ở một vai trò duy nhất. Vai trò candidate cho phép đọc kết quả matching của chính mình. Quyền quản lý doanh nghiệp dựa trên membership đã xác minh, không suy ra chỉ từ cạnh works_at. Người giới thiệu xử lý yêu cầu gửi trực tiếp cho mình; quản trị viên thực hiện các tác vụ quản lý được chỉ định và có audit.

Bảng 2.1: Tác nhân và quyền nghiệp vụ chính

Tác nhân	Chức năng	Điều kiện quyền
Ứng viên	Hồ sơ, kỹ năng, tìm việc, gợi ý, ứng tuyển, gửi/hủy referral	Tài nguyên của chính mình; job và path hợp lệ
Người giới thiệu	Nhận, chấp nhận, từ chối, xác nhận, hoàn tất yêu cầu	Yêu cầu gửi tới mình; employment phù hợp
Quản lý tuyển dụng	Quản lý jobs và đơn ứng tuyển	Membership đúng doanh nghiệp đã xác minh; consent
Quản trị viên	Danh mục kỹ năng, tài khoản, xác minh membership	Quyền admin; ghi audit; không mặc định đọc mọi CV



Hình 2.1: Các nhóm chức năng theo tác nhân

2.2. YÊU CẦU CHỨC NĂNG

Hệ thống phải cho phép đăng ký, đăng nhập, quản lý phiên và chỉnh sửa hồ sơ. Kỹ năng được chọn từ danh mục để tạo has_skill, kèm mức thành thạo và số năm kinh nghiệm. Các thiết lập discoverable, allow_in_paths và quyền chia sẻ phần hồ sơ ảnh hưởng trực tiếp tới dữ liệu được trả về, không chỉ tới việc ẩn thành phần trên giao diện.

Tìm việc phải đọc tin còn mở, hỗ trợ bộ lọc và phân trang. Matching trả về danh sách xếp hạng cùng điểm thành phần, các kỹ năng khớp và kỹ năng bắt buộc còn thiếu. Chế độ matched_only chỉ giữ job có ít nhất một kỹ năng thỏa mức yêu cầu, khác với eligible là điều kiện đủ required skills và kinh nghiệm tối thiểu theo dữ liệu đã lưu.

Quản lý kết nối gồm gửi lời mời, chấp nhận, từ chối, chặn và xóa theo quy tắc quyền của hai bên. Tìm đường giới thiệu sử dụng các cạnh accepted, người dùng active, quyền chia sẻ hợp lệ và người nhận có employment current/verified. Referral phải có vòng đời rõ ràng, giữ lịch sử và chống tạo trùng khi người dùng gửi lại sau khi mất phản hồi.

Bảng 2.2: Yêu cầu và tiêu chí nghiệm thu cốt lõi

Mã	Yêu cầu	Tiêu chí có thể kiểm chứng
F01	Hồ sơ và kỹ năng được lưu thật	Đọc lại API thấy giá trị; tồn tại edge has_skill
F02	Gợi ý cá nhân hóa	Thêm/xóa kỹ năng làm đổi kết quả; có explanation
F03	Đường giới thiệu có nguồn dữ liệu	AQL trả vertices/edges/hops từ graph thật
F04	Tôn trọng trạng thái và privacy	Chặn cạnh hoặc tắt allow_in_paths làm mất nhánh
F05	Người giới thiệu hợp lệ	Employment chưa xác minh hoặc hết hạn bị loại
F06	Workflow referral nhất quán	Lịch sử pending → accepted → submitted → completed
F07	Gửi lại không tạo trùng	Cùng idempotency key/payload trả record cũ
F08	Ghi nhiều collections nguyên tử	Lỗi audit làm notification trước đó rollback

2.3. YÊU CẦU PHI CHỨC NĂNG

Đầu vào API phải được kiểm tra kiểu, độ dài và tập giá trị cho phép. Actor được lấy từ xác thực phía server; client không thể đổi actor_id để đọc gợi ý của người khác. DTO graph không trả email, token hoặc các thuộc tính riêng tư. Câu truy vấn cố định và bind variables tách dữ liệu đầu vào khỏi mã AQL.

Ứng dụng phải khởi động được bằng Docker Compose, có health checks, giữ dữ liệu qua persistent volumes và có hướng dẫn backup/restore. Backend tổ chức theo route, schema, service, repository để có thể kiểm thử độc lập ở mức module nhưng vẫn tích hợp với database thật. Giới hạn depth và tài nguyên là một phần của yêu cầu vận hành cục bộ.

Về khả năng sử dụng, giao diện tiếng Việt thể hiện rõ trạng thái tải, lỗi, dữ liệu rỗng và thao tác thử lại. Điểm hiển thị trên thang 100 cần kèm giải thích rằng đó là điểm xếp hạng. Các kiểm tra browser và accessibility hỗ trợ phát hiện lỗi ở những luồng đã chạy, chưa thay thế khảo sát người dùng thực tế.

2.4. PHÂN TÍCH CÁC CA SỬ DỤNG

2.4.1. Xem công việc được gợi ý

Tiền điều kiện là ứng viên đã đăng nhập và có hồ sơ. Người dùng mở dashboard hoặc trang tìm việc ở chế độ gợi ý. Backend đọc actor, các cạnh kỹ năng và tin còn mở, áp dụng bộ lọc trước khi sort và phân trang. Hậu điều kiện là danh sách có điểm và explanation; trường hợp chưa có kỹ năng phù hợp được thể hiện bằng trạng thái rỗng hoặc hướng dẫn bổ sung hồ sơ.

Một lỗi đã được ghi nhận trong quá trình phát triển là dashboard trước đây chỉ đọc các tin mới nhất mặc dù hồ sơ đã có kỹ năng. Bản sửa bổ sung thành phần RecommendedJobs và view /jobs?view=recommended, gọi /matching/jobs. Điều này cho thấy có thuật toán phía database chưa đủ: giao diện phải thực sự sử dụng endpoint xếp hạng mới đáp ứng nhu cầu gợi ý [8].

2.4.2. Tìm đường tới người giới thiệu

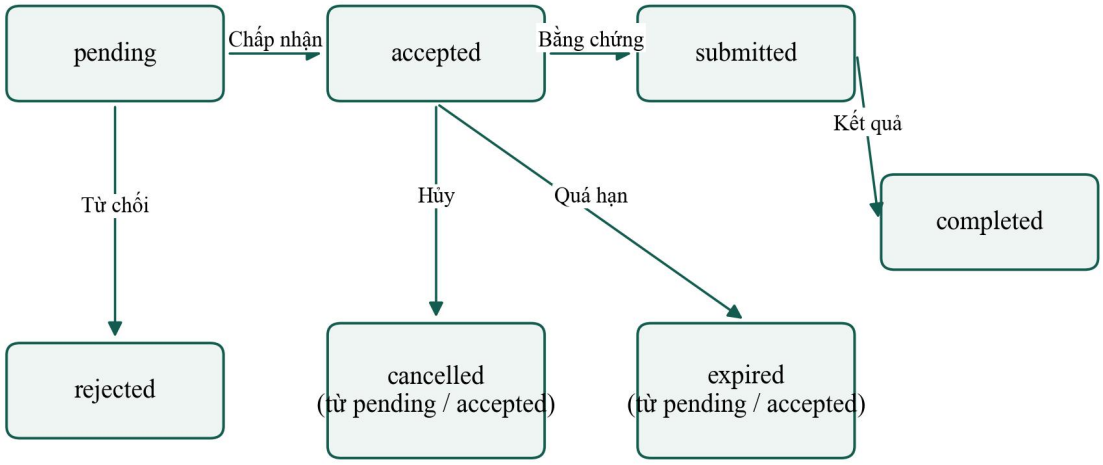
Tiền điều kiện là job còn mở và thuộc doanh nghiệp hợp lệ. Hệ thống xác định các nhân viên current/verified tại doanh nghiệp, duyệt mạng lưới từ ứng viên với depth từ 1 tới 3, rồi chỉ giữ các đường có endpoint trong nhóm nhân viên được phép xem. Kết quả gồm danh sách đỉnh, cạnh, số bước, người nhận và giải thích điểm đường.

Nếu không có endpoint hợp lệ, hệ thống trả danh sách rỗng. Đối với các công ty public chưa có nhân viên được xác minh trong ứng dụng, đây là kết quả đúng. Không tạo cạnh works_at giả để làm đầy màn hình; bộ dữ liệu kiểm thử riêng được sử dụng để trình diễn trường hợp có đường đi.

2.4.3. Gửi và xử lý yêu cầu giới thiệu

Ứng viên chọn referrer từ đường đã tìm và gửi request với idempotency key. Service kiểm tra lại tài khoản, job, employment, path, consent và quota trong transaction. Nếu điều kiện thay đổi so với lúc xem đường, thao tác bị từ chối. Khi hợp lệ, referral, notification và audit được ghi đồng thời.

Người nhận chỉ được thực hiện các chuyển trạng thái cho phép. accepted chưa có nghĩa đã giới thiệu; submitted cần evidence_ref và completed cần outcome. evidence_ref là một reference do người giới thiệu nhập trong MVP, chưa chứng minh hồ sơ đã được gửi sang một hệ thống bên ngoài.



Hình 2.2: Vòng đời yêu cầu giới thiệu trong MVP

2.5. QUY TẮC NGHIỆP VỤ VÀ TRƯỜNG HỢP BIÊN

Một kết nối lưu một cặp canonical endpoints; cạnh chiều ngược không tạo quan hệ mới. Pending, rejected, blocked và deleted không được dùng để tìm đường. Người cùng doanh nghiệp hoặc cùng kỹ năng không tự động là bạn. Cycles không được tạo thành đường lặp đỉnh; đường thay thế vẫn phải được giữ để có thể chọn khi nhánh chính bị loại.

Một application duy nhất cho mỗi candidate/job; một referral duy nhất cho mỗi candidate/referrer/job trong vòng đời MVP, kể cả khi đã terminal. Quota yêu cầu giới thiệu được giới hạn tối đa ba người nhận cho một job/candidate và mười requests/ngày theo tài liệu nghiệp vụ. Thao tác ghi có revision sử dụng If-Match; revision cũ trả lỗi thay vì ghi đè im lặng.

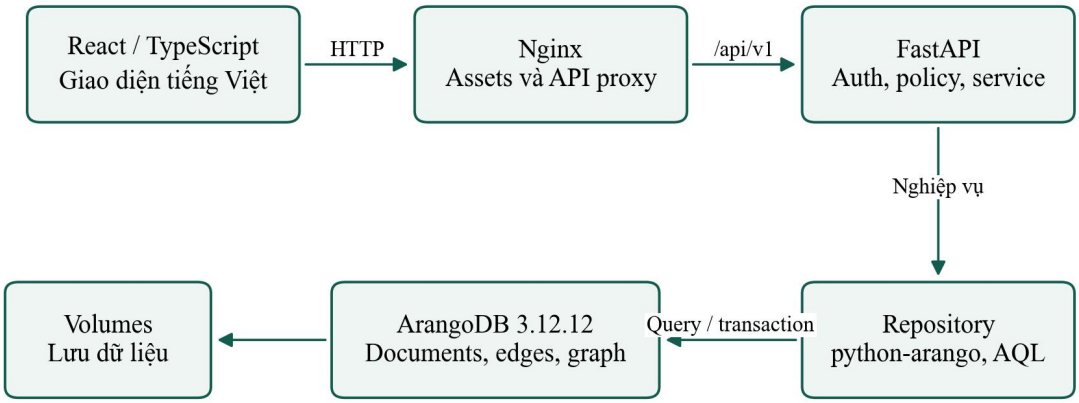
Job đóng hoặc công ty inactive bị loại khỏi matching. Kỹ năng không còn active không được tính như kỹ năng hợp lệ. Job không có yêu cầu kỹ năng không tự nhận điểm kỹ năng tối đa. Thiếu min_experience_years ở tin public cho điểm kinh nghiệm bằng 0, nhưng không tự tạo một ngưỡng kinh nghiệm bắt buộc. Những quy tắc này bảo vệ sự minh bạch của kết quả khi dữ liệu không đầy đủ.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1. KIẾN TRÚC TỔNG THỂ

Ứng dụng sử dụng kiến trúc modular monolith. React đảm nhiệm giao diện, điều hướng và quản lý dữ liệu phía browser; Nginx phục vụ assets và proxy; FastAPI xác thực, kiểm tra quyền và xử lý nghiệp vụ; repositories dùng python-arango để gọi ArangoDB. React được tổ chức theo components và trạng thái giao diện [5]; FastAPI cung cấp API cùng schema/OpenAPI [4], còn driver python-arango là lớp truy cập database [6].

Frontend không kết nối trực tiếp tới ArangoDB. Khi người dùng mở gợi ý, frontend gửi HTTP request, backend xác định actor, service gọi repository, repository thực thi AQL và trả DTO được giới hạn trường. Ranh giới này cho phép kiểm soát quyền trên server dù người dùng gọi endpoint bên ngoài giao diện.



Hình 3.1: Kiến trúc ứng dụng và luồng dữ liệu

3.1.1. Phân tách module

Backend gồm auth, users, profiles, connections, companies, jobs, skills, applications, referrals, graph, matching, notifications, admin và audit. Mỗi module có api.py, schemas.py, service.py, repository.py. Quy tắc chung về lỗi, dependencies, policy, transactions và cấu hình nằm trong backend/core. Các truy vấn được chia thành file AQL và các phần include cố định để tái sử dụng.

Frontend nhóm theo features tương ứng. TanStack Query đọc/cache API theo phiên và hỗ trợ invalidation khi hồ sơ hoặc kỹ năng thay đổi. React Hook Form và Zod xử lý form; React Flow hiển thị các đỉnh/cạnh do API trả về. Việc vẽ đồ thị không tự thực hiện tìm đường: các đường đã được AQL tính trước khi đến giao diện.

3.2. THIẾT KẾ DỮ LIỆU ARANGODB

3.2.1. Collections thực thể và nghiệp vụ

Bốn collections trung tâm là users, companies, jobs và skills. Các collections applications, referral_requests, notifications và audit_logs hỗ trợ workflow. Collections xác thực, phiên, idempotency và quota được tách để không đưa thông tin nhạy cảm vào DTO graph. Sự tách biệt này giúp mô hình quan hệ nghề nghiệp không bị lẫn với dữ liệu xác thực.

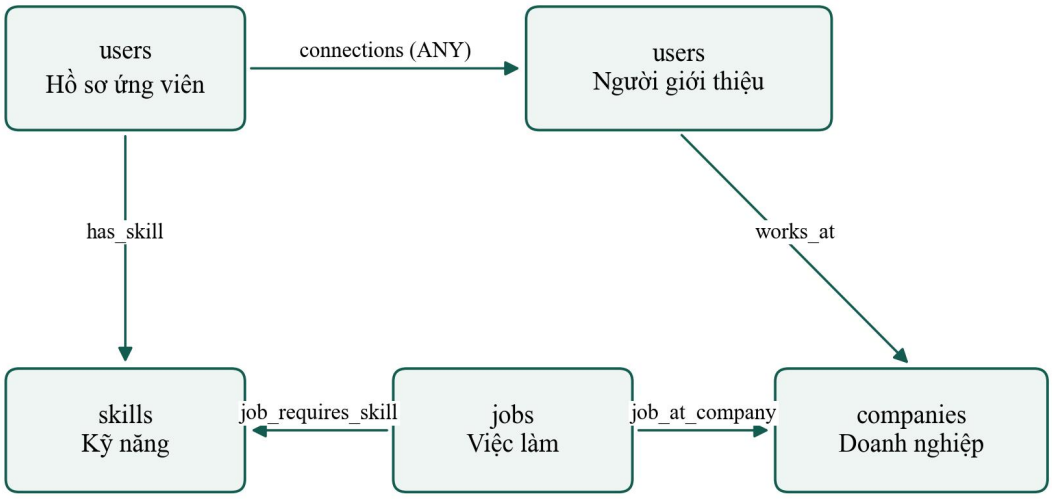
Bảng 3.1: Các collections chính

Collection	Nội dung tiêu biểu	Vai trò
users	display_name, roles, profile, privacy, status	Hồ sơ và actor
companies	name, slug, members, status, source	Doanh nghiệp và membership
jobs	title, min_experience_years, location, level, work_mode, source	Tin tuyển dụng
skills	name, normalized_name, status, source	Danh mục kỹ năng
applications	candidate_id, job_id, status, consent, resume_ref	Đơn ứng tuyển
referral_requests	candidate_id, referrer_id, job_id, status, history	Yêu cầu giới thiệu
notifications	recipient_id, event_key, payload, read_at	Thông báo
audit_logs	actor_id, resource_id, action, request_id	Dấu vết nghiệp vụ

3.2.2. Edge collections và chiều quan hệ

Bảng 3.2: Định nghĩa các quan hệ đồ thị

Edge collection	Đỉnh nguồn → đỉnh đích	Thuộc tính/ý nghĩa
connections	users → users	status, accepted_at, deleted_at, trust; duyệt ANY
works_at	users → companies	status, verified, start_at, end_at; employment
has_skill	users → skills	proficiency, years_experience
job_requires_skill	jobs → skills	kind, importance, min_proficiency, min_years
job_at_company	jobs → companies	Mỗi job có đúng một employer



Hình 3.2: Mô hình graph cốt lõi của Job Referral Network

Named graph mang tên jobReferralGraph. _key là khóa trong collection, _id có dạng collection/key, _from và _to tham chiếu _id. Trong HTTP, ứng dụng sử dụng key và chuyển sang _id tại service; client không được tự cung cấp endpoints tùy ý để tạo quan hệ không thuộc quyền của mình.

Quan hệ kỹ năng được tách khỏi profile text. Headline "Backend Developer" không tự tạo has_skill; người dùng phải chọn và lưu kỹ năng. Cách này giảm nhập nhằng khi đối chiếu, nhưng chất lượng gợi ý phụ thuộc vào mức độ đầy đủ và trung thực của dữ liệu tự khai báo.

3.2.3. Schema, migration và chỉ mục

Migration có version, checksum và kiểm tra drift. Các migration 001–005 lần lượt xây nền tảng graph, indexes, backend, indexes cho listings và provenance của nguồn public. Bootstrap có thể chạy lại mà không reset dữ liệu; schema không tương thích phải được phát hiện. Thay đổi dữ liệu cần backup và phương án phục hồi, không mặc định mọi migration có thể đảo ngược an toàn.

Bảng 3.3: Một số chỉ mục quan trọng

Index	Trường chính	Mục đích
connection_pair_unique	pair_key, unique	Chống trùng cạnh kể cả chiều ngược
connections_out_status / in_status	_from hoặc _to, status, deleted_at	Traversal và kiểm tra trạng thái
company_current_employees	_to, status, verified, end_at	Chọn nhân viên theo doanh nghiệp
user_skill_unique	_from, _to, unique	Một kỹ năng trên mỗi người dùng
job_skill_unique	_from, _to, unique	Một yêu cầu kỹ năng trên mỗi job
job_company_uniqu e	_from, unique	Một employer cho mỗi job
jobs_open_recent	status, published_at	Lọc tin mở và sắp xếp
referral_triplet_uniq ue	candidate_id, referrer_id, job_id	Chống trùng yêu cầu giới thiệu

Index nguồn job dùng unique sparse cho provenance optional theo migration 005. Khi trường source vắng mặt, không được suy luận từ unique index rằng bản ghi có nguồn xác minh. Bộ kiểm tra integrity còn đối chiếu các quan hệ và schema; tên index trong thiết kế cần được kiểm chứng bằng EXPLAIN ở truy vấn quan trọng.

3.3. THIẾT KẾ THUẬT TOÁN GỢI Ý VIỆC LÀM

3.3.1. Đầu vào và tiền xử lý

Đầu vào gồm profile của actor, các edges has_skill, job còn mở, company active và các edges job_requires_skill. Một kỹ năng được coi là khớp khi tồn tại trong danh mục active, ứng viên có edge tới kỹ năng đó, proficiency không thấp

hơn min_proficiency và years_experience không thấp hơn min_years. Tin được lọc theo tham số q/location/work_mode/level/company_id trước khi phân trang.

Các yêu cầu được chia required và preferred, mỗi yêu cầu có importance. Với mỗi nhóm, ratio bằng tổng importance của kỹ năng khớp chia tổng importance của nhóm. Nếu cả hai nhóm có dữ liệu, điểm kỹ năng K = 0,8 × R + 0,2 × P; nếu chỉ có một nhóm thì dùng ratio của nhóm đó; nếu không có requirements thì K = 0. Trọng số 0,8/0,2 phản ánh ưu tiên thiết kế cho yêu cầu bắt buộc, chưa được hiệu chỉnh bằng dữ liệu tuyển dụng thực tế.

3.3.2. Công thức điểm tổng hợp

Điểm gợi ý việc làm được định nghĩa S = 0,40K + 0,20E + 0,15L + 0,15V + 0,10W. K là điểm kỹ năng; E là điểm kinh nghiệm; L là điểm địa điểm; V là điểm cấp độ; W là điểm hình thức làm việc. Các thành phần nằm trong khoảng [0,1]. Giao diện có thể biểu diễn 100 × S để dễ đọc nhưng không đổi ý nghĩa của phép đo.

Nếu min_experience_years có giá trị dương, E = min(1, experience_years/min_experience_years); nếu bằng 0 thì E = 1; nếu thông tin này chưa biết thì E = 0. L = 1 khi job remote hoặc địa điểm bằng đúng giá trị trong profile. V = 1 khi cấp độ bằng nhau. W = 1 khi hình thức job nằm trong các lựa chọn của ứng viên. Các phép khớp đơn giản này chưa xử lý khoảng cách địa lý hoặc độ tương đồng giữa các cấp độ.

Bảng 3.4: Thành phần và trọng số điểm việc làm

Thành phần	Trọng số	Cách diễn giải
Kỹ năng K	0,40	Required/preferred có importance, xét proficiency và years
Kinh nghiệm E	0,20	Tỷ lệ đáp ứng số năm; thiếu dữ liệu tin thì 0
Địa điểm L	0,15	Remote hoặc location bằng nhau
Cấp độ V	0,15	Level khớp chính xác
Hình thức W	0,10	Work mode thuộc preference

Salary_overlap chỉ là thông tin giải thích khi đủ dữ liệu, không được lén bổ sung vào S. Eligible kiểm tra không thiếu required skill và đáp ứng min

experience nếu có. Eligible không chứng nhận đáp ứng mọi yêu cầu trong tin gốc, vì dữ liệu được chuẩn hóa có thể chỉ chứa một phần nội dung tuyển dụng.

3.3.3. Ví dụ tính điểm và xếp hạng

Ví dụ minh họa công thức: ứng viên khớp 3/4 tổng importance của required và toàn bộ preferred, nên K = 0,8 × 0,75 + 0,2 × 1 = 0,8. Nếu E = 1, L = 1, V = 1 và W = 0 thì S = 0,32 + 0,20 + 0,15 + 0,15 = 0,82. Điểm 82/100 này là ví dụ tính toán, không phải một kết quả đo tuyển dụng hoặc khẳng định xác suất 82%.

Truy vấn job_matches sắp xếp theo score giảm dần, published_at giảm dần rồi job._id tăng dần để ổn định thứ tự khi dữ liệu không thay đổi. matched_only=true lọc trước sort và LIMIT. Offset pagination có thể dịch trang khi dữ liệu thay đổi giữa hai requests; hệ thống chưa bảo đảm một snapshot chung cho toàn bộ lượt duyệt.

3.3.4. Dữ liệu thiếu và nguồn công khai

Các kỹ năng trích từ tin công khai được lưu như tín hiệu preferred nếu chưa có căn cứ xác định yêu cầu bắt buộc. Lương, cấp độ hoặc kinh nghiệm chưa rõ được giữ thiếu/unspecified thay vì điền giá trị giả. Cách này làm một số tin có điểm thấp hơn nhưng giúp kết quả có thể giải thích. Bổ sung thông tin phải giữ provenance và không quy kết rằng dữ liệu tự chuẩn hóa là xác nhận của nhà tuyển dụng.

3.4. THIẾT KẾ TÌM VÀ XẾP HẠNG ĐƯỜNG GIỚI THIỆU

3.4.1. Duyệt graph có ràng buộc

Truy vấn bắt đầu từ actor và duyệt 1..max_depth qua connections của jobReferralGraph bằng ANY. PRUNE loại cạnh không accepted, chưa có accepted_at, đã deleted, đỉnh inactive hoặc deleted, quyền discoverable/allow_in_paths không hợp lệ và các người dùng nằm trong blocked_ids. OPTIONS giới hạn edgeCollections=[connections], vertexCollections=[users], dùng bfs và uniqueVertices=path.

Unique vertices theo path loại chu trình trong từng đường nhưng vẫn giữ đường thay thế. Nếu dùng global deduplication, một đỉnh được thăm qua đường đầu có thể làm mất đường khác cần để xếp hạng. Endpoint được lọc theo nhân viên current/verified, thời gian việc làm và quyền visibility hợp lệ tại doanh nghiệp. Hops chỉ đếm các cạnh connections, không tính cạnh works_at vào số bước kết nối xã hội.

3.4.2. Điểm đường và đường ngắn nhất

Gọi h là số hops và T là trust trung bình của các cạnh, điểm đường D = 0,60T + 0,40 × (1/h). Trust thiếu được mặc định 0,5. Điểm ưu tiên cân bằng độ tin cậy khai báo và độ ngắn của đường; trust trong fixture là dữ liệu giả lập và chưa được hiệu chỉnh để dự báo thành công.

Chế độ xếp hạng thông thường sort D giảm dần rồi hops tăng dần, path_key tăng dần. Chế độ shortest_only ưu tiên hops trước D. Vì vậy đường được gợi ý đầu tiên không nhất thiết là đường có ít cạnh nhất. Dự án tìm đường ngắn nhất trong tập đường hợp lệ bằng traversal có ràng buộc và sort, không tuyên bố đây là thuật toán weighted shortest path tối ưu cho graph lớn.

3.4.3. Kết hợp việc làm và mạng lưới

jobs_with_referral giữ các job eligible, tìm best_path hợp lệ cho từng job rồi sort job score trước path score. Cách sắp xếp này tránh đưa một công việc thiếu kỹ năng lên đầu chỉ vì có người quen. Gợi ý việc làm thông thường vẫn hoạt động khi không tồn tại referral path, nên người dùng mới chưa có mạng lưới vẫn có thể xem công việc phù hợp.

Chi phí traversal có thể tăng theo số nhánh và độ sâu; với b nhánh trung bình, một mô tả gần đúng là O(b^d) cho phần mở rộng đường. Khi thực hiện tìm path trên nhiều jobs, phần tính toán có thể bị lặp. Giới hạn d ≤ 3 giảm phạm vi nhưng không bảo đảm hiệu năng cho graph dày hoặc catalog lớn; chương 5 trình bày số liệu và giới hạn phép đo.

3.5. THIẾT KẾ API, GIAO DIỆN VÀ TÍNH NHẤT QUÁN

Bảng 3.5: Các API chính cho gợi ý và graph

Endpoint dưới /api/v1	Chức năng	Đầu ra/điều kiện
GET /matching/jobs	Danh sách gợi ý theo hồ sơ	Page[MatchResult], chỉ candidate của chính mình
GET /jobs/{job_id}/matching	Điểm một công việc	score, eligible, explanation
GET /graph/referral-paths	Tìm đường giới thiệu	vertices, edges, hops, referrer, path_score

Endpoint dưới /api/v1	Chức năng	Đầu ra/điều kiện
GET /graph/jobs/{job_id} /opportunities	Cơ hội có đường giới thiệu	Job eligible và best_path
POST /referral-requests	Tạo yêu cầu	Kiểm tra path hiện tại, idempotency và transaction

Dashboard hiển thị "Gợi ý theo hồ sơ của bạn". Trang tìm việc có view recommended và bộ lọc. Trang đường giới thiệu hiển thị graph, danh sách đường, score, hops và lý do. Khi thêm/xóa kỹ năng hoặc sửa profile, frontend invalidate dữ liệu matching để lần đọc tiếp theo sử dụng hồ sơ mới; không thay score bằng phép tính riêng không đồng bộ ở browser.

ETag/If-Match kiểm soát cập nhật dựa trên revision. Idempotency key chống tạo trùng khi retry. Stream transaction khóa các collections liên quan, kiểm tra lại policy và ghi workflow đồng thời. Thiết kế khóa rộng trong MVP giúp tính nhất quán nhưng có thể tuần tự hóa nhiều ghi; cần đo throughput và thu hẹp phạm vi khóa trước khi mở rộng.

CHƯƠNG 4: CÀI ĐẶT HỆ THỐNG

4.1. MÔI TRƯỜNG VÀ CÔNG NGHỆ

Môi trường thao tác của dự án là Windows với PowerShell, Docker Desktop chạy Linux containers và mã nguồn tại D:\bigdata. Các phiên bản trong bảng lấy từ compose.yaml, pyproject.toml và frontend/package.json được đọc khi biên soạn, không phải khẳng định đây là các phiên bản mới nhất ngoài dự án. Các dependency khóa phục vụ việc tái lập môi trường.

Bảng 4.1: Cấu hình công nghệ theo mã nguồn

Thành phần	Phiên bản/cấu hình	Nguồn
ArangoDB	3.12.12, single server	compose.yaml
Python	Yêu cầu ≥ 3.10; image backend Python 3.12	pyproject.toml, Dockerfile
FastAPI / Pydantic	0.143.0 / 2.14.0	pyproject.toml
python-arango	8.3.6	pyproject.toml
React / TypeScript	19.3.0 / 5.9.3	frontend/package.json
Vite / React Flow	8.3.4 / 12.12.0	frontend/package.json
Node.js	Yêu cầu ≥ 22.12.0	frontend/package.json
Kiểm thử	Pytest, Playwright, axe; Ruff/TypeScript/build	Manifests và scripts

4.2. CÀI ĐẶT DATABASE VÀ KHỞI TẠO

Trước tiên tạo virtual environment, cài dependencies từ requirements.lock và cài dự án ở chế độ editable. scripts/init_env.py tạo cấu hình secrets ngẫu nhiên khi cần. Sau đó khởi động riêng ArangoDB và chờ health check đạt; chạy bootstrap để áp dụng migration, seed để tạo fixture nếu cần và provision_runtime.py để tạo runtime database user cho API.

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.lock
.venv\Scripts\python.exe -m pip install --no-deps -e .
.venv\Scripts\python.exe scripts/init_env.py
docker compose up -d --wait arangodb
.venv\Scripts\python.exe -m database.cli bootstrap
.venv\Scripts\python.exe -m database.cli seed
.venv\Scripts\python.exe scripts/provision_runtime.py
docker compose --profile web up -d --build --wait
```

Các lệnh trên phục vụ khởi tạo một môi trường phù hợp; không yêu cầu tạo lại secrets trên môi trường đã có cấu hình. Bootstrap không xóa dữ liệu cũ. Với database đang hoạt động, áp dụng migration theo quy trình dừng writers, backup, cập nhật và verify. Không xóa volume để xử lý lỗi cấu hình.

Database chính mặc định là job_referral. Frontend ở http://127.0.0.1:5173, Swagger ở http://127.0.0.1:8000/docs và giao diện ArangoDB ở http://127.0.0.1:8529. Các cổng chỉ bind loopback theo Compose. Tài khoản operator database khác tài khoản ứng dụng; credentials không được đưa vào báo cáo hoặc slide.

4.3. HIỆN THỰC AQL VÀ MATCHING

database/queries.py giữ danh sách tên truy vấn cho phép, mở rộng include theo tên file cố định và bind tham số. Những giới hạn ở repository gồm max_depth 1–3, limit 1–100, offset 0–1000, q tối đa 160 ký tự và location tối đa 120 ký tự. Timestamp dùng UTC phía server cho điều kiện employment; client không được thay thời điểm để lách policy.

MatchingRepository sử dụng GraphQueries, chọn job_matches hoặc jobs_with_referral. MatchingService kiểm tra role candidate, chuyển job key thành jobs/key và trả items, offset, limit, has_more. Đọc limit+1 hỗ trợ xác định còn trang sau mà không cần trả toàn bộ tập kết quả.

Phần dưới trích cấu trúc traversal từ database/aql/path_rows.aql; employees và blocked_ids được chuẩn bị ở phần truy vấn liên quan. Đây là đoạn giải thích cơ chế, không phải truy vấn độc lập để chạy thiếu các bind variables và subqueries.

```
FOR v, e, p IN 1..@max_depth ANY @actor_id
 GRAPH "jobReferralGraph"
 PRUNE invalid =
  (e != null AND (e.status != "accepted"
    OR e.accepted_at == null OR e.deleted_at != null))
  OR v == null OR v.status != "active"
  OR v.deleted_at != null
  OR (v._id != @actor_id AND
    (!v.privacy.discoverable OR !v.privacy.allow_in_paths))
  OR v._id IN blocked_ids
 OPTIONS { order: "bfs", uniqueVertices: "path",
  uniqueEdges: "path", edgeCollections: ["connections"],
  vertexCollections: ["users"] }
 FILTER !invalid
```

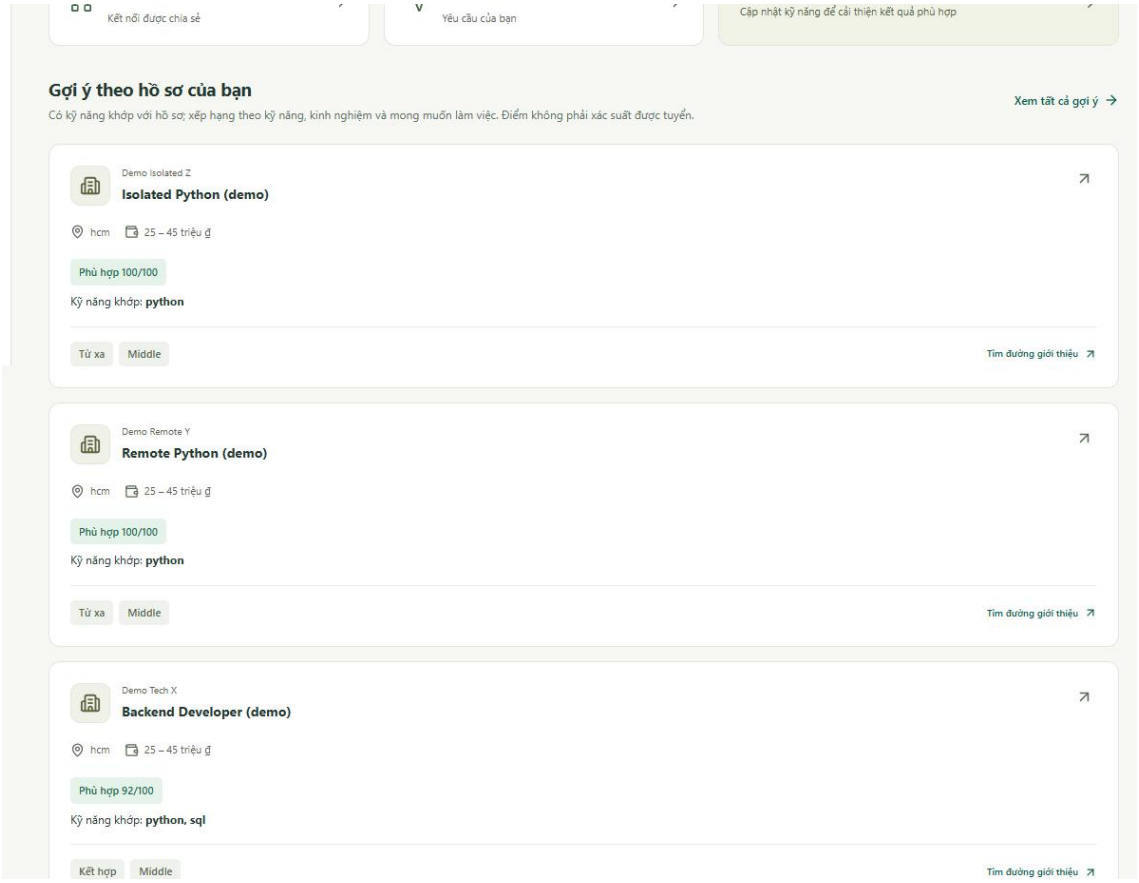
Guard e != null xử lý trường hợp PRUNE được xét tại root. FILTER !invalid loại node tại điểm dừng, vì chỉ PRUNE không đồng nghĩa mọi giá trị tại nhánh đó đã bị loại khỏi kết quả. Việc giới hạn collections giúp traversal không nhầm đường user → skill → user là một quan hệ quen biết.

Điểm tổng hợp được tính trong matching_rows.aql theo các weights truyền từ repository. Các tham số trọng số được kiểm tra hữu hạn, không âm và tổng bằng 1. Source salary thiếu được giữ null trong explanation. Dữ liệu trả về chỉ gồm các trường job công khai và thông tin gợi ý được schema cho phép.

4.4. HIỆN THỰC GIAO DIỆN VÀ LUỒNG NGƯỜI DÙNG

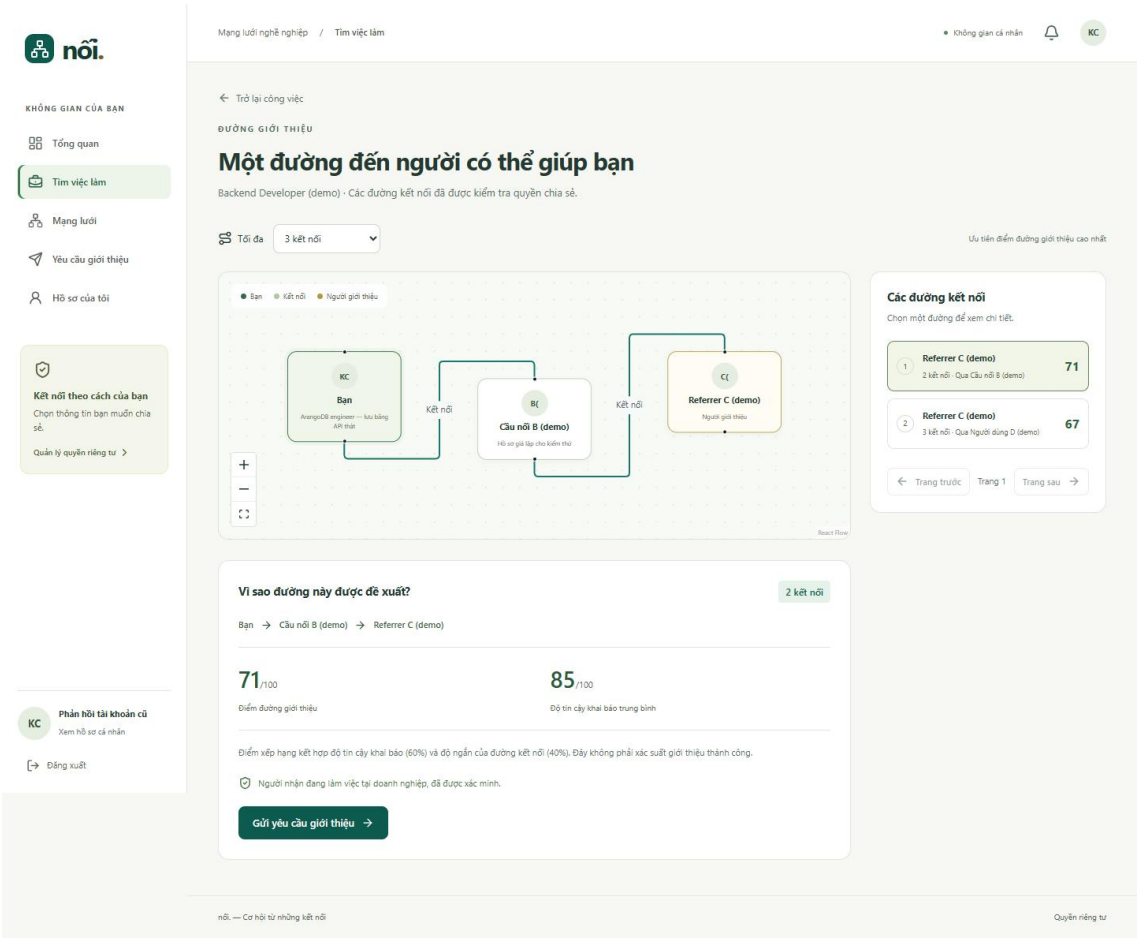
Frontend dùng React Router cho điều hướng, TanStack Query để gọi API và quản lý cache. Dashboard, JobsPage và RecommendedJobs sử dụng endpoint matching cho danh sách gợi ý. ProfilePage có đường dẫn tới trang gợi ý; kỹ năng phải được lưu qua API trước khi tính điểm thay đổi. Bộ lọc được truyền xuống server để áp dụng trước pagination.

Ảnh dưới lấy từ artifact browser của dự án. Tên công ty và hồ sơ trong ảnh là dữ liệu kiểm thử; ảnh không chứng minh có nhân viên thật tại các công ty public. Nội dung dashboard thể hiện điểm và kỹ năng khớp, giúp phân biệt danh sách gợi ý với danh sách tin mới.



Hình 4.1: Khu vực gợi ý việc làm trên dashboard với dữ liệu kiểm thử

Trang PathsPage đọc referral paths và vẽ bằng GraphView/React Flow. Người dùng chọn số kết nối tối đa, xem các đường thay thế, số hops và điểm đường. Thành phần giải thích ghi rõ trust khai báo và độ ngắn được sử dụng để xếp hạng; nút gửi referral nối kết quả phân tích graph với quy trình nghiệp vụ.



Hình 4.2: Giao diện đường giới thiệu A → B → C trên fixture

Giao diện có phiên riêng cho từng actor trong browser tests. Thao tác accept của B được ứng viên A đọc lại qua API; thao tác complete của C được phản ánh ở lịch sử của A. Browser test cố ý làm mất phản hồi sau commit để kiểm tra retry; dữ liệu workflow vẫn được đọc từ backend/database thật.

4.5. DỮ LIỆU CÔNG KHAI VÀ NGUỒN GỐC

Catalog public_jobs_2026-10-12.json ghi nhận 29 tin kỹ thuật tại Việt Nam: KMS Technology 5, Bosch Group 12 và Grab 12, cùng 28 kỹ năng trong danh mục. Các số này là quy mô snapshot nhập vào dự án, không phải toàn bộ tin tuyển dụng của các công ty và không phải xác nhận các tin vẫn mở tại ngày đọc báo cáo [9].

Mỗi job/company/skill có thông tin source phù hợp, như URL, publisher, external_id và retrieved_at. Collector và importer được tách: collector lấy nguồn công khai, importer kiểm tra quan hệ và ghi trong transaction. Mô tả job là tóm tắt; báo cáo không chép toàn văn quảng cáo tuyển dụng. Các lương hoặc kinh nghiệm chưa có trong nguồn không được tự điền.

Các listings giả lập cũ được archive và đổi nhãn kiểm thử. Công ty public dùng IDs riêng, không thêm nhân viên giả hoặc quyền membership cho tài khoản test. Vì vậy danh mục thật dùng để chứng minh nguồn tin và matching, còn graph fixture dùng để chứng minh đường đi. Việc nhập lặp cùng snapshot được thiết kế để giữ revisions khi nội dung không đổi.

4.6. BẢO MẬT, GHI NHẬN VÀ VẬN HÀNH CỤC BỘ

Xác thực sử dụng JWT và Argon2id; refresh tokens có rotation và cơ chế thu hồi family. Runtime database user không có quyền DDL, khác với operator thực hiện migrations. Policy được kiểm tra tại dependencies và service; các thao tác nhạy cảm được kiểm tra lại trong transaction để tránh sử dụng quyền đã thay đổi.

Containers có giới hạn tài nguyên, root filesystem chỉ đọc và bỏ capabilities theo cấu hình. Backend và Nginx chạy OS user không root; image ArangoDB hiện vẫn có OS UID 0 với capabilities bị bỏ theo báo cáo vận hành. Secrets nằm ngoài source/image; logs giữ request ID, route template, status và duration thay vì token hoặc body nhạy cảm.

Khi triển khai cục bộ, health/ready kiểm tra database thật. Không auto-migrate khi API start. Backup dùng arangodump và restore vào database mới bằng arangorestore, sau đó verify schema, integrity và kết quả query. Các bước bảo vệ backup, TLS, lịch purge, quản lý secrets và vận hành sản xuất cần được hoàn thiện trước khi đưa lên môi trường công khai [10].

CHƯƠNG 5: THỬ NGHIỆM VÀ TRIỂN KHAI

5.1. PHƯƠNG PHÁP VÀ MÔI TRƯỜNG THỬ NGHIỆM

Kiểm thử tích hợp dùng ArangoDB thật trên Docker và tạo database UUID riêng cho từng lượt. Fixtures là dữ liệu nghiệp vụ giả lập, nhưng repository, AQL và transaction là thật. Browser tests kết nối FastAPI qua HTTP TCP và runtime user không root; không thay kết quả nghiệp vụ bằng repository mock. Sau lượt chạy, scripts dọn đúng tài nguyên UUID thuộc lượt đó.

Các kết quả bên dưới được tổng hợp từ artifact đã có trong dự án tại thời điểm biên soạn. Báo cáo không cộng 21 tests chuyên sâu vào 167 tests regression như hai tập độc lập, vì các tập có thể chồng lặp. Số tests pass thể hiện kịch bản đã kiểm tra, không quy đổi thành phần trăm chất lượng sản phẩm hay độ chính xác gợi ý.

5.2. KẾT QUẢ KIỂM THỬ CHỨC NĂNG

Bảng 5.1: Tổng hợp kết quả kiểm thử lưu trong dự án

Bộ kiểm tra	**Số ca**	**Fail/Error/ Skip**	**Thời gian**	**Artifact**
Python regression sau sửa gợi ý	167	0 / 0 / 0	533,522 s	recommendations -regression.xml
Browser xác minh gợi ý và luồng chính	20	0 / 0 / 0	124,387 s	recommendations -verified-playwright.xml
Graph/index/transaction chuyên sâu	21	0 / 0 / 0	141,334 s	arango-proof-tests.xml

Python regression được ghi ngày 12/10/2026. Nhóm kiểm tra matching bao gồm thêm/xóa kỹ năng, hồ sơ rỗng, filter trước pagination, tìm kiếm theo tên skill, giới hạn input và cách ly actor. Nhóm graph bao gồm chiều ANY, depth, chu trình, cạnh không hợp lệ, quyền riêng tư và employment. Nhóm workflow kiểm tra các chuyển trạng thái, idempotency, quota và phân quyền.

Browser suite kiểm tra việc lưu kỹ năng, hiển thị danh sách gợi ý, bộ lọc, mobile, loading/error/retry và luồng giới thiệu nhiều actor. Báo cáo sửa gợi ý ghi một lượt trước đó có 19 pass và 1 fail do fixture sử dụng email .test bị validation

từ chối; fixture được sửa và chạy lại toàn bộ đạt 20 pass [8]. Việc giữ thông tin lượt thất bại giúp kết quả cuối có bối cảnh, thay vì chỉ công bố lượt thành công.

Các kiểm tra Ruff, TypeScript, Vite build và Prettier được ghi PASS trong báo cáo sửa gợi ý. Đây là bằng chứng từ lượt phát triển tương ứng, không phải lời khẳng định rằng chúng đã được chạy lại khi tạo tài liệu Word này. Kết quả cần được chạy lại nếu mã nguồn hoặc môi trường thay đổi trước buổi bảo vệ.

5.3. CHỨNG MINH ĐÓNG GÓP CỦA ARANGODB

Script prove_arangodb.py tạo database jrn_proof_<uuid>, áp dụng migration/seed và gọi GraphQueries thật. Baseline có hai đường: A → B → C với 2 hops và score 0,71; A → D → E → C với 3 hops và score khoảng 0,6733. Referrer C có employment current/verified tại công ty kiểm thử. Đó là các kết quả lưu trong arango-behavior-proof.json [11].

Bảng 5.2: Thay đổi dữ liệu và kết quả graph quan sát được

Thao tác trong database kiểm thử	**Kết quả đọc lại**	**Nội dung được chứng minh**
Baseline	Hai đường 2 hops và 3 hops	Đường tìm từ cạnh thật
Chặn cạnh B–C	Chỉ còn đường 3 hops	Status cạnh tác động traversal
B tắt allow_in_paths	Chỉ còn đường 3 hops	Privacy loại người trung gian
Employment của C chưa xác minh	Danh sách rỗng	Kiểm tra endpoint theo nghiệp vụ
Ghi audit sai schema sau notification	Lỗi schema 1620; rollback=true	Transaction không để partial writes

EXPLAIN ghi nhận TraversalNode và IndexNode; các index names gồm company_current_employees, connections_in_status, connections_out_status và edge. PROFILE của một query fixture nhỏ ghi execution_time khoảng 0,00813 giây. Kết quả này xác nhận cơ chế thực thi trong trường hợp đã kiểm tra, không dùng để suy ra latency ở graph lớn hoặc workload đồng thời.

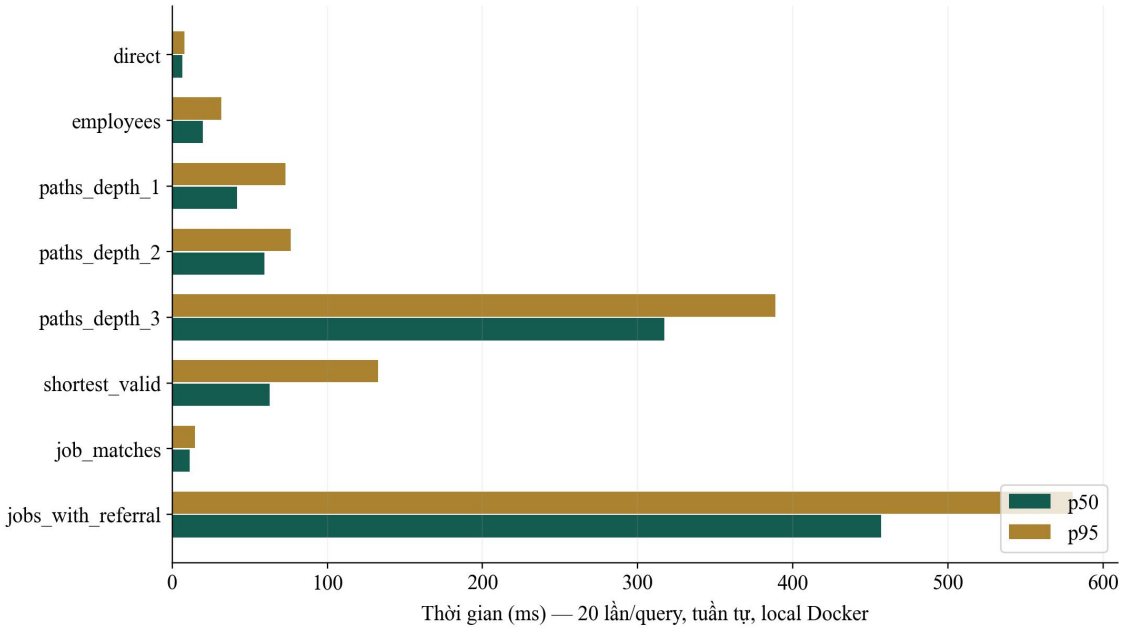
Database kiểm thử được dọn sau khi thực hiện, và main_database_modified=false trong artifact. Sự tách biệt này cho phép tái hiện bằng chứng mà không cần thay đổi quan hệ của người dùng thật. Kịch bản có thể trình bày trực tiếp bằng lệnh python scripts/prove_arangodb.py trong virtual environment khi ArangoDB đang hoạt động.

5.4. ĐÁNH GIÁ HIỆU NĂNG TRUY VẤN

Benchmark đã lưu dùng ArangoDB 3.12.12 single server trên Docker, dữ liệu synthetic với 1.134 users, 1.128 connections và 1.012 works_at. Mạng lưới bổ sung có branching 10, depth 3; mỗi loại query chạy 20 lần tuần tự trên môi trường local. Các p50/p95 dưới đây là số liệu trong benchmark.json, được làm tròn ba chữ số thập phân [12].

Bảng 5.3: Thời gian truy vấn trong benchmark local

Query	Số lần	p50 (ms)	p95 (ms)	Lỗi
direct	20	6,555	7,946	0
employees	20	19,863	31,653	0
paths_depth_1	20	41,925	73,029	0
paths_depth_2	20	59,359	76,511	0
paths_depth_3	20	317,302	388,701	0
shortest_valid	20	62,855	132,623	0
job_matches	20	11,193	14,485	0
jobs_with_referral	20	456,868	580,469	0



Hình 5.1: So sánh p50 và p95 truy vấn trên benchmark synthetic local

Trong phép đo này, việc tăng depth từ 2 lên 3 làm tăng đáng kể latency, phù hợp với việc phải mở rộng nhiều nhánh. jobs_with_referral có thời gian cao hơn job_matches vì bổ sung tìm đường cho công việc. Nhận xét chỉ áp dụng cho

dataset và cấu hình đã đo; bảng không chứng minh độ phức tạp thực nghiệm trên mọi topology.

Benchmark được tạo ở giai đoạn nền tảng trước các cập nhật catalog và giao diện gợi ý ngày 12/10/2026. Vì vậy nó được dùng làm số liệu tham khảo cho thiết kế truy vấn, không thay thế benchmark của catalog public hiện tại. Không có phép đo SQL/Neo4j đối chứng, không đo concurrency, cluster, SLA hoặc khả năng chịu tải của toàn ứng dụng. Cần thực hiện lại khi quy mô jobs, số kết nối hoặc cấu hình máy chủ thay đổi.

5.5. TRIỂN KHAI VÀ PHỤC HỒI

Docker Compose triển khai ba services: frontend, backend và arangodb. Frontend phụ thuộc backend healthy; backend phụ thuộc arangodb healthy. Persistent volumes giữ data/apps. Cấu hình networking tách proxy network và không cho frontend cùng network database. API startup kiểm tra schema/index/graph drift; operator thực hiện migration trước khi restart writers.

Artifact recovery.json ngày 12/10/2026 ghi ArangoDB 3.12.12, schema=pass, integrity=pass và source_unchanged=true sau restore vào database tạm do script tạo. Counts của snapshot phục hồi gồm 35 jobs, 6 companies và 28 skills; đây là tổng dữ liệu gồm cả kiểm thử/lưu trữ, không đồng nhất với 29 jobs public còn mở. Path_hops rỗng trong nguồn và bản restore là một kết quả đối chiếu hợp lệ, không chứng minh có đường giới thiệu ở database chính.

Hướng dẫn vận hành yêu cầu verify sau restore rồi mới chuyển cấu hình ứng dụng. Chưa có RPO/RTO cam kết, backup offsite định kỳ hoặc kiểm thử phục hồi dưới tải sản xuất. Việc restore local thành công là bằng chứng khôi phục được snapshot đã thử, không phải nghiệm thu toàn bộ kế hoạch dự phòng.

5.6. ĐỐI CHIẾU MỤC TIÊU VÀ GIỚI HẠN ĐÁNH GIÁ

Bảng 5.4: Mức đáp ứng mục tiêu đề tài

Mục tiêu	Trạng thái	Bằng chứng/giới hạn
ArangoDB document và edge schema	Đã triển khai và kiểm thử	Migrations, named graph, verify
Gợi ý việc làm theo hồ sơ	Đã triển khai và kiểm thử	AQL score/explanation; API/browser regression

Mục tiêu	Trạng thái	Bằng chứng/giới hạn
Tìm đường giới thiệu có ràng buộc	Đã triển khai và kiểm thử	Fixture 2/3 hops; block/privacy/employment
Workflow giới thiệu xuyên UI/API/DB	Đã triển khai và kiểm thử	Nhiều actor, history, idempotency
Nguồn công khai và provenance	Đã triển khai cho snapshot	29 jobs; chưa tự đồng bộ đóng tin
Độ chính xác gợi ý với người dùng thật	Chưa đánh giá	Chưa có labels, Precision@K/NDCG hoặc khảo sát
Tải lên CV, email và ATS	Chưa triển khai	Chỉ lưu reference/xác nhận trong MVP
Vận hành production và cluster	Chưa nghiệm thu	TLS, image findings, load/ops còn mở

Kết quả đáp ứng trọng tâm đề tài ứng dụng ArangoDB cho gợi ý và graph có thể giải thích. Tuy nhiên, tests xác nhận công thức và điều kiện lọc không đồng nghĩa người dùng sẽ đánh giá thứ hạng hữu ích. Để đo chất lượng gợi ý, cần tập đánh giá có nhãn phù hợp, lựa chọn K, baseline và các chỉ số như Precision@K hoặc NDCG, đồng thời có quy trình thu thập consent và bảo vệ dữ liệu.

Audit security-summary.json còn ghi production_ready=false [10]. Các image findings mức HIGH, TLS và điều kiện vận hành cần được xử lý/kiểm tra lại; không dùng việc tests pass để bỏ qua những điều kiện đó. Báo cáo không gán tỷ lệ "hoàn thiện 100%" vì còn các mục tiêu ngoài MVP chưa thực hiện.

5.7. KỊCH BẢN TRÌNH DIỄN KHI BẢO VỆ

Trình diễn bắt đầu bằng documents/edges trong ArangoDB và tên jobReferralGraph. Tiếp theo mở dashboard, lưu kỹ năng rồi xem gợi ý thay đổi; đối chiếu request /matching/jobs với score/explanation. Sau đó dùng fixture riêng để hiển thị đường A → B → C và phân biệt các cạnh connections với works_at.

Để làm rõ đóng góp database, chạy prove_arangodb.py và giải thích baseline, chặn B–C, tắt quyền của B, bỏ xác minh employment của C, EXPLAIN và rollback. Cuối cùng trình diễn referral từ pending tới completed bằng các phiên actor riêng. Tin công khai được trình bày như dữ liệu nguồn; graph fixture được trình bày như kiểm thử tích hợp, không nhập nhằng hai nhóm.

Các hướng dẫn chi tiết nằm trong docs/demo-guide.md và docs/arango-proof-guide.md. Chuẩn bị trước gồm kiểm tra Docker healthy, database.cli verify và chạy lại các ca chọn lọc. Nếu môi trường không chạy, sử dụng artifact lưu trước và ghi rõ đó là kết quả đã thu thập, không giả lập một lượt demo trực tiếp.

KẾT LUẬN

1. KẾT QUẢ ĐẠT ĐƯỢC

Đề tài đã xây dựng Job Referral Network với React, FastAPI và ArangoDB làm cơ sở dữ liệu cốt lõi. Thực thể được lưu bằng document, quan hệ bằng edge và quản lý qua named graph. AQL được sử dụng để đối chiếu kỹ năng, tính điểm việc làm và duyệt các đường kết nối có ràng buộc. Kết quả trả về có giải thích và liên kết tới quy trình giới thiệu.

Các kiểm thử lưu trong dự án chứng minh chức năng hoạt động trên database thật: thay đổi kỹ năng ảnh hưởng gợi ý; block/privacy/employment ảnh hưởng đường đi; transaction lỗi được rollback; retry không tạo trùng. Các bộ regression Python và browser được xác minh đạt 167 và 20 ca trong lượt báo cáo tương ứng. Các artifact và hướng dẫn tái lập giúp đối chiếu kết quả từ thiết kế tới ứng dụng đã cài đặt.

2. ĐÁNH GIÁ KẾT QUẢ

Trọng tâm ứng dụng ArangoDB đã được đáp ứng ở mức MVP chạy cục bộ, có source và bằng chứng kiểm thử. Hệ thống sử dụng quan hệ đồ thị cho nghiệp vụ thay vì chỉ dùng ArangoDB như kho JSON. Việc tách job score khỏi path score giúp phản ánh hai câu hỏi riêng: công việc phù hợp đến đâu và người dùng có thể tiếp cận referrer thế nào.

Mức hoàn thiện được đánh giá theo các mục tiêu và giới hạn tại Bảng 5.4, không quy đổi thành một tỷ lệ không có phương pháp đo. Tài liệu này mô tả kết quả kỹ thuật đã có; chưa xác nhận hiệu quả tuyển dụng thực tế hay khả năng triển khai công khai ở quy mô lớn.

3. HẠN CHẾ

Trọng số gợi ý còn được lựa chọn theo quy tắc; dữ liệu kỹ năng/kinh nghiệm phần lớn tự khai báo và chưa có nhãn để hiệu chỉnh. Location và level dùng phép khớp đơn giản. Catalog public là snapshot và có trường chưa đầy đủ; các công ty public chưa có nhân viên được xác minh để tạo đường giới thiệu thật trong ứng dụng.

Depth giới hạn ba hops và transaction khóa rộng có thể làm giảm khả năng mở rộng. Benchmark chỉ chạy tuần tự local trên synthetic dataset, chưa phản ánh catalog lớn hoặc tải đồng thời. Upload CV, email, ATS, đồng bộ đóng tin và một

số quy trình vận hành vẫn ngoài phạm vi. Production readiness còn chưa đạt theo audit đã lưu.

4. HƯỚNG PHÁT TRIỂN

Hướng thứ nhất là cải thiện dữ liệu: đồng bộ trạng thái tin theo nguồn, chuẩn hóa location/level tốt hơn và có cơ chế xác minh employment, kỹ năng phù hợp. Hướng thứ hai là thu thập phản hồi có consent để đánh giá thứ hạng bằng tập kiểm tra độc lập, so sánh baseline và hiệu chỉnh trọng số; chỉ thêm học máy khi dữ liệu và tiêu chí đánh giá cho phép.

Hướng thứ ba là tối ưu truy vấn và vận hành: benchmark lại trên nhiều topology, đo concurrency, giảm tính toán path lặp và thu hẹp lock scope mà vẫn giữ kiểm tra policy nhất quán. Hướng cuối cùng là hoàn thiện file storage, email/ATS, TLS, secrets management, backup offsite và diễn tập phục hồi. Mỗi mở rộng cần acceptance criteria và bằng chứng mới trước khi được ghi nhận là hoàn thành.

TÀI LIỆU THAM KHẢO

[1] ArangoDB, “Graphs”, tài liệu ArangoDB 3.12. https://docs.arango.ai/arangodb/3.12/graphs/ . Truy cập ngày 12/10/2026.

[2] ArangoDB, “AQL Documentation”, tài liệu ArangoDB 3.12. https://docs.arango.ai/arangodb/3.12/aql/ . Truy cập ngày 12/10/2026.

[3] ArangoDB, “Graph traversals in AQL”, tài liệu ArangoDB 3.12. https://docs.arango.ai/arangodb/3.12/aql/graph-queries/traversals/ . Truy cập ngày 12/10/2026.

[4] FastAPI, tài liệu chính thức. https://fastapi.tiangolo.com/ . Truy cập ngày 12/10/2026.

[5] React, “Quick Start”, tài liệu chính thức. https://react.dev/learn . Truy cập ngày 12/10/2026.

[6] python-arango, tài liệu driver Python. https://docs.python-arango.com/en/main/ . Truy cập ngày 12/10/2026.

[7] Job Referral Network, mã nguồn và manifests tại D:\bigdata: backend/, database/, frontend/, compose.yaml. Kiểm tra ngày 12/10/2026.

[8] Job Referral Network, “Sửa việc hồ sơ đã lưu kỹ năng nhưng chưa thấy gợi ý”, docs/reports/recommendations-fix-2026-10-12.md và các JUnit liên quan, 12/10/2026.

[9] Job Referral Network, catalog public_jobs_2026-10-12.json và docs/public-data.md, snapshot nguồn công khai ngày 12/10/2026.

[10] Job Referral Network, security-summary.json, security-review-2026-10-11.md và recovery.json trong docs/reports/, 11–12/10/2026.

[11] Job Referral Network, arango-behavior-proof.json và arango-proof-tests.xml trong docs/reports/, bằng chứng graph/transaction ngày 12/10/2026.

[12] Job Referral Network, docs/reports/benchmark.json, benchmark local ghi cho ngày 11/10/2026; đọc lại ngày 12/10/2026.

PHỤ LỤC: HƯỚNG DẪN TÁI LẬP VÀ ĐỐI CHIẾU

A. KIỂM TRA CHẠY CỤC BỘ

Chạy các lệnh từ thư mục dự án khi Docker Desktop và virtual environment đã sẵn sàng. Các kiểm thử tạo database UUID riêng trên ArangoDB thật; cần operator credentials để tạo/dọn các database này. Không thay bằng mocks khi database mất kết nối.

```
docker compose ps
.venv\Scripts\python.exe -m database.cli verify
.venv\Scripts\python.exe scripts/prove_arangodb.py
.venv\Scripts\python.exe -m pytest -q
```

Để chạy browser theo hướng dẫn của dự án, sử dụng scripts/check_frontend.py. Script tạo backend/Vite ở ports riêng và fixtures riêng. Chế độ debug có thể dùng trước buổi trình diễn để quan sát từng bước; giữ bản report của lượt chạy và ghi thời điểm nếu kết quả được đưa vào báo cáo sửa đổi.

B. BẢN ĐỒ SOURCE VÀ BẰNG CHỨNG

Bảng PL.1: Đối chiếu nội dung báo cáo với source/artifact

Nội dung	File hoặc thư mục
Công thức gợi ý	database/aql/matching_rows.aql; database/queries.py
Sort/filter/pagination matching	database/aql/job_matches.aql; backend/modules/matching/
Traversal và privacy	database/aql/path_rows.aql; backend/modules/graph/
Gợi ý có đường giới thiệu	database/aql/jobs_with_referral.aql
Schema, named graph và indexes	database/migrations/001–005; docs/database/
Giao diện gợi ý	frontend/src/features/jobs/RecommendedJobs.tsx; JobsPage.tsx
Giao diện graph	frontend/src/features/network/PathsPage.tsx; GraphView.tsx
Workflow	backend/modules/referrals/; backend/core/transactions.py
Kiểm thử	tests/; frontend/e2e/features.spec.ts; docs/reports/
Triển khai	compose.yaml; Dockerfile; frontend/Dockerfile

C. CÁC THÔNG TIN HÀNH CHÍNH CẦN HOÀN THIỆN

Tên ngành, loại báo cáo chính thức, danh sách sinh viên, mã sinh viên, lớp và giảng viên hướng dẫn chưa được cung cấp khi biên soạn. Các vị trí tương ứng trên bìa giữ nhãn “[Cần bổ sung]” để tác giả kiểm tra và điền theo thông tin được nhà trường xác nhận. Không sử dụng tên tài khoản máy tính để suy ra họ tên tác giả.

Lời cảm ơn trong báo cáo sử dụng cách diễn đạt chung; tác giả cần kiểm tra tên cá nhân/tập thể và nội dung đóng góp trước khi nộp. Báo cáo không bổ sung kết quả phỏng vấn, xác nhận doanh nghiệp, chữ ký hay chứng nhận thử nghiệm của bên thứ ba khi chưa có bằng chứng.